User guide

This system formalizes mathematical results stated in natural language and verifies their proofs in the Lean 4 proof assistant. Its use requires no programming experience and no prior knowledge of Lean.

1 Overview

A user supplies a statement and, optionally, an informal proof. A group of AI agents then proceeds through the following stages.

1. **Formalization.** The orchestrator translates the statement into Lean. An auditor, which does not see the source text, reconstructs the statement from the translation, and the two are compared.
2. **Planning.** The orchestrator decomposes the proof into a sequence of lemmas.
3. **Proof.** Provers address the lemmas in parallel. A lemma that resists repeated attempts returns to the orchestrator, which may prove it, divide it, or revise the plan.
4. **Assembly.** The lemmas are combined into a single proof, which Lean verifies in full.
5. **Review and reporting.** An independent reviewer compares the verified proof with the source statement, and the system then prepares a report in mathematical language.

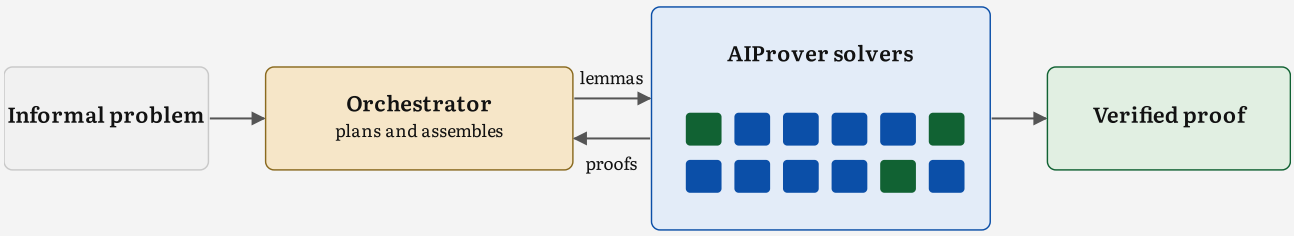


Figure 1. System architecture. The orchestrator assigns lemmas to a pool of solvers. Green squares denote successful sessions.

Figure 2 presents these stages in order. Each green label denotes a check that must pass before the run proceeds, and each dashed arrow denotes the stage to which a failed check returns.

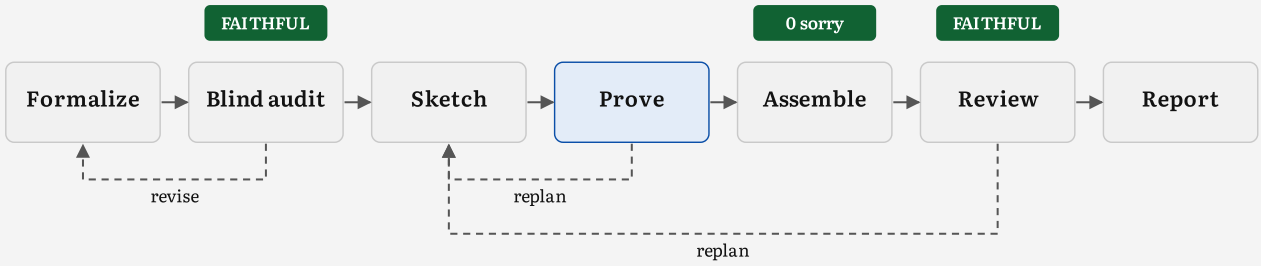


Figure 2. Stages of a run. A report is produced only after the proof passes Lean and both faithfulness checks.

> **Scope of verification.** Lean establishes that the proof is correct for the formal statement. It cannot establish that the formal statement expresses the intended theorem; the agents assess this correspondence, and their assessment may be mistaken. Consequently, the formal statement in the report should be read directly before a result is relied upon.

2 Submitting a problem

The home page leads to each section and shows the run that most needs attention. The **Runs** page lists all runs, with the most recent first. Figure 3 identifies its principal elements.

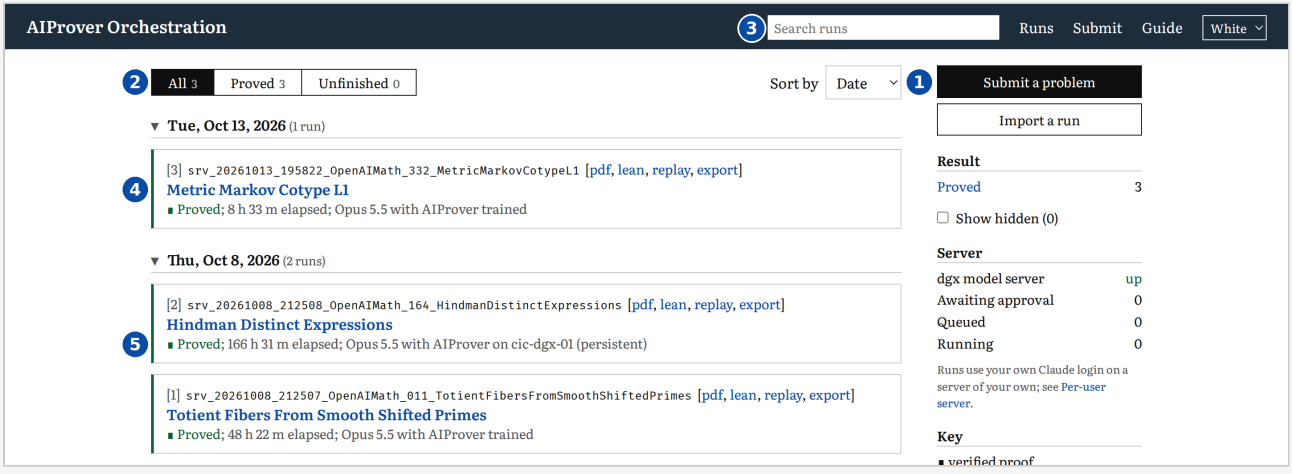


Figure 3. The Runs page.
1 Submit a problem.
2 Filter runs by outcome.
3 Search by name.
4 Name of a run, which opens its record.
5 Result of the run.

Step 1: Problem

Select **Submit a problem**, then enter the statement and, where available, an informal proof. LaTeX notation is accepted, as Figure 4 illustrates. The run name is optional and serves to identify the run later.

Run name: sum_of_odd_numbers

Statement: For every positive integer n, the sum of the first n odd positive integers equals n^2; that is, $\sum_{k=1}^{n}(2k-1) = n^2$.

Informal proof: By induction on n. For $n = 1$ both sides equal 1. If the claim holds for n, adding $2(n+1) - 1 = 2n + 1$ to both sides gives $n^2 + 2n + 1 = (n+1)^2$.

1. **Problem** 2. Library 3. Models and settings 4. Review

Run name optional; becomes the run identifier

sum_of_odd_numbers

Source

Informal statement

Statement

For every positive integer n, the sum of the first n odd positive integers equals n^2; that is, $\sum_{k=1}^{n} (2k-1) = n^2$.

132 characters · ~33 tokens

Informal proof optional

By induction on n. For $n=1$ both sides equal 1. If the claim holds for n, adding $2(n+1)-1 = 2n+1$ to both sides gives $n^2 + 2n + 1 = (n+1)^2$.

149 characters · ~37 tokens

Group optional; sorts related problems together

No group

Cancel Continue

Figure 4. The first step of a submission, completed with the example above.

The optional **Group** field associates related problems. A user may select an existing group, create a new group, or leave the field empty. Groups are specific to each server. On the main page, **Sort by** arranges runs by date or by group, and any date or group may be collapsed by selecting its heading.

Step 2: Library, models, and settings

The default values are suitable for most runs, and these steps may therefore be left unchanged.

Step 3: Review

The final step summarizes the submission, as Figure 5 shows. After **Submit** is selected, the run enters the queue and continues independently of the page.

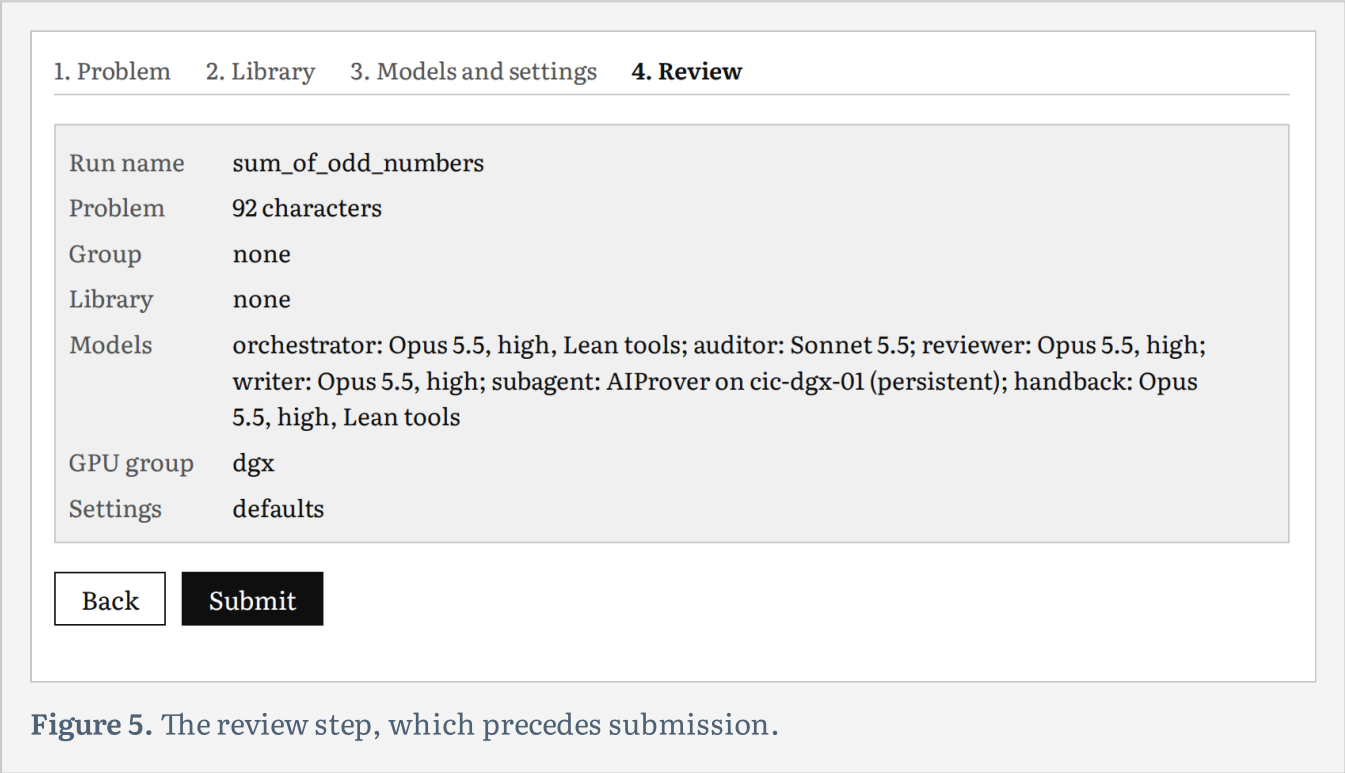


Figure 5. The review step, which precedes submission.

3 Formulating a statement

The agents interpret a statement literally. Consequently, an omission that a colleague would resolve from context may lead to an unintended formalization.

Ambiguous	Precise
Let G be a group. Then G has a normal subgroup of index 2.	Let G be a finite group of order $2m$ with m odd. Then G has a normal subgroup of index 2.
Let f be continuous. Then f attains its maximum.	Let $f : [0,1] \to \mathbb{R}$ be continuous. Then there is $x \in [0,1]$ with $f(y) \leq f(x)$ for all $y \in [0,1]$.

- **State every hypothesis,** including those conventional in the field.
- **Define specialized terms and notation.** Standard objects require no definition, whereas a notion introduced in a particular paper does.
- **Provide a proof where one exists,** even in outline, since it guides the plan.
- **Identify cited results** and state their sources.

4 Monitoring a run

A run typically requires several hours, and difficult problems may require days. The page need not remain open.

Selecting the name of a run opens its record. The **Progress** tab, shown in Figure 6, lists the steps of the run in chronological order.

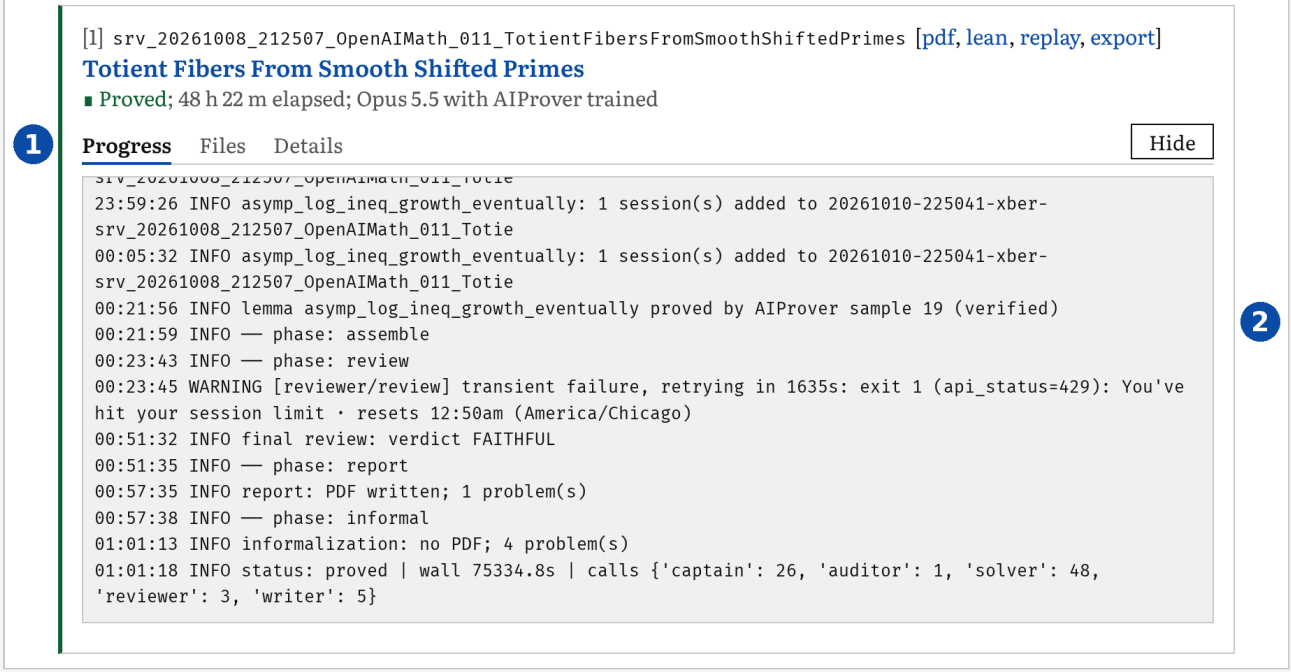


Figure 6. An open run.
1 Tabs of the record.
2 Progress log, here from the final lemma through assembly, review, and reporting.

5 Interpreting results

Each run concludes with one of the following results.

Result	Meaning
∎ Proved	Lean has verified a complete proof of the formal statement.
¬ Disproved	Lean has verified a proof of the negation. The hypotheses should be examined, since an omitted hypothesis can produce this result.
○ Lemmas unproved	Some lemmas of the plan remain without proofs.
× Failed	The run ended before reaching a proof.
/ Cancelled	The run was stopped by a user.

The **Files** tab, shown in Figure 7, describes each output of the run. The informal proof is the recommended starting point, and the report is the appropriate place to compare the formal statement with the intended theorem.

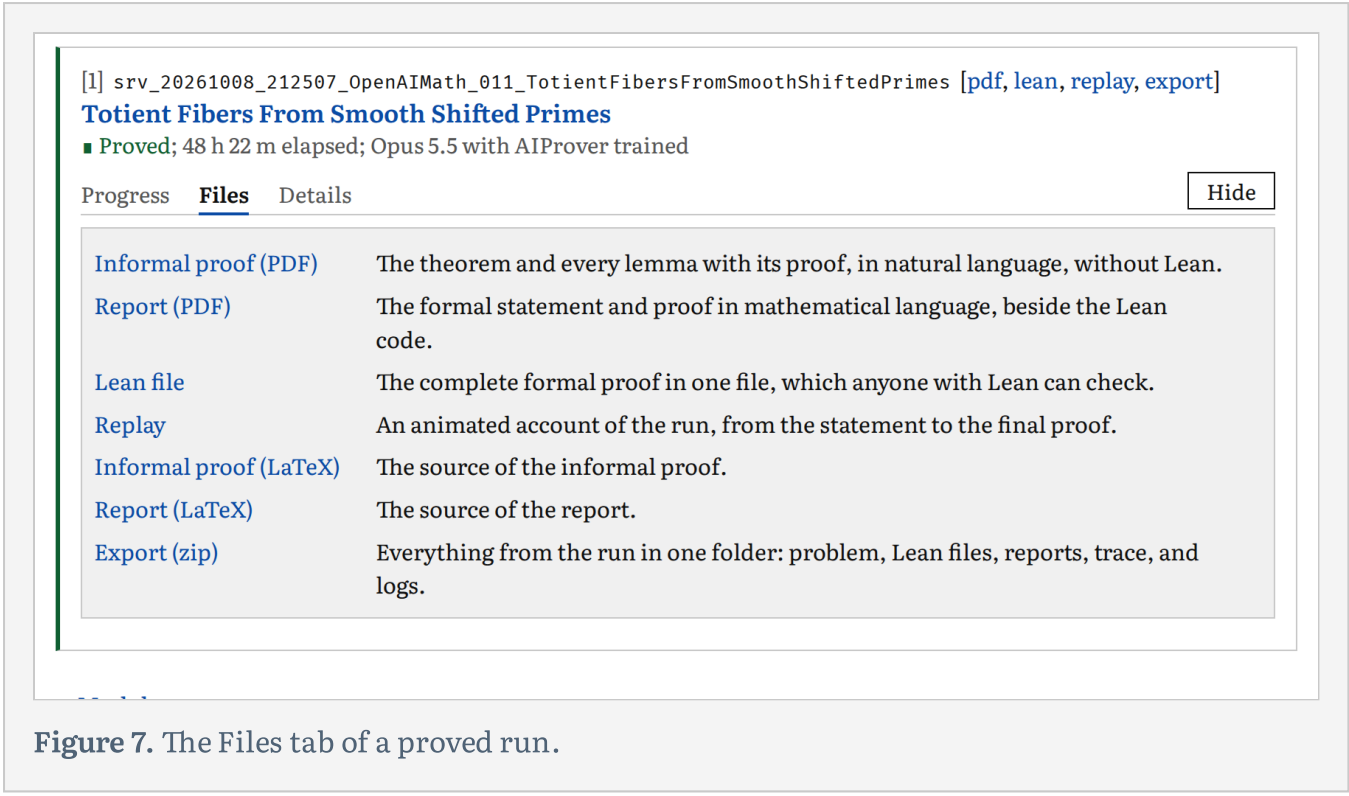
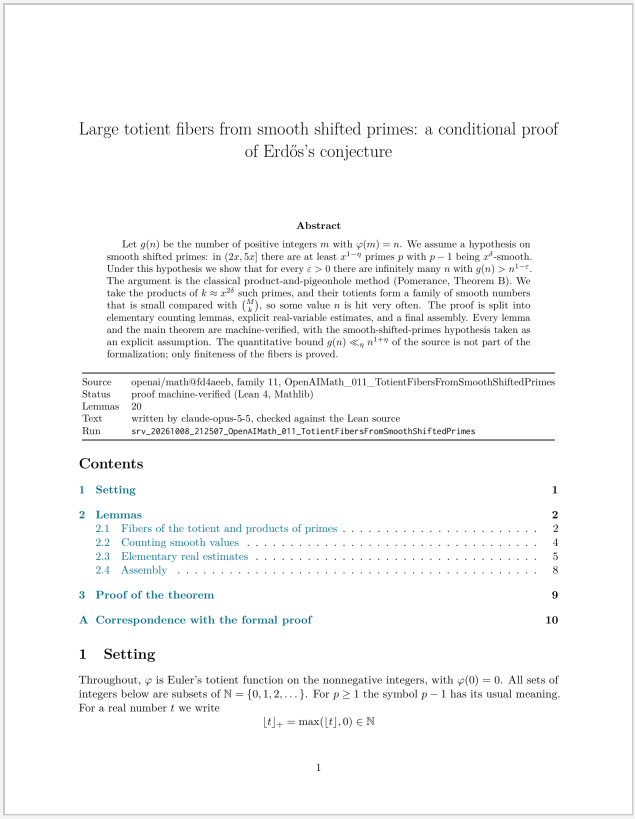


Figure 7. The Files tab of a proved run.

Figure 8 reproduces the first page of each document for one proved run. The informal proof states the theorem and every lemma in natural language, whereas the report sets each statement and proof beside its Lean code. Figure 9 compares the two treatments of a single lemma.



Large totient fibers from smooth shifted primes: a conditional proof of Erdős's conjecture

Abstract

Let $g(n)$ be the number of positive integers m with $\varphi(m) = n$. We assume a hypothesis on smooth shifted primes: in $(2x, 5x]$ there are at least $x^{1-\eta}$ primes p with $p-1$ being x^{δ}-smooth. Under this hypothesis we show that for every $\varepsilon > 0$ there are infinitely many n with $g(n) > n^{1-\varepsilon}$. The argument is the classical product-and-pigeonhole method (Pomerance, Theorem B). We take the products of $k \approx x^{2\delta}$ such primes, and their totients form a family of smooth numbers that is small compared with $\binom{M}{k}$, so some value n is hit very often. The proof is split into elementary counting lemmas, explicit real-variable estimates, and a final assembly. Every lemma and the main theorem are machine-verified, with the smooth-shifted-primes hypothesis taken as an explicit assumption. The quantitative bound $g(n) \ll_\eta n^{1+\eta}$ of the source is not part of the formalization; only finiteness of the fibers is proved.

Source	openai/math@fd4aeeb, family 11, OpenAIMath_011_TotientFibersFromSmoothShiftedPrimes
Status	proof machine-verified (Lean 4, Mathlib)
Lemmas	20
Text	written by claude-opus-5-5, checked against the Lean source
Run	srv_20261008_212507_OpenAIMath_011_TotientFibersFromSmoothShiftedPrimes

Contents

1 Setting 1

2 Lemmas 2
2.1 Fibers of the totient and products of primes . . . 2
2.2 Counting smooth values . . . 4
2.3 Elementary real estimates . . . 5
2.4 Assembly . . . 8

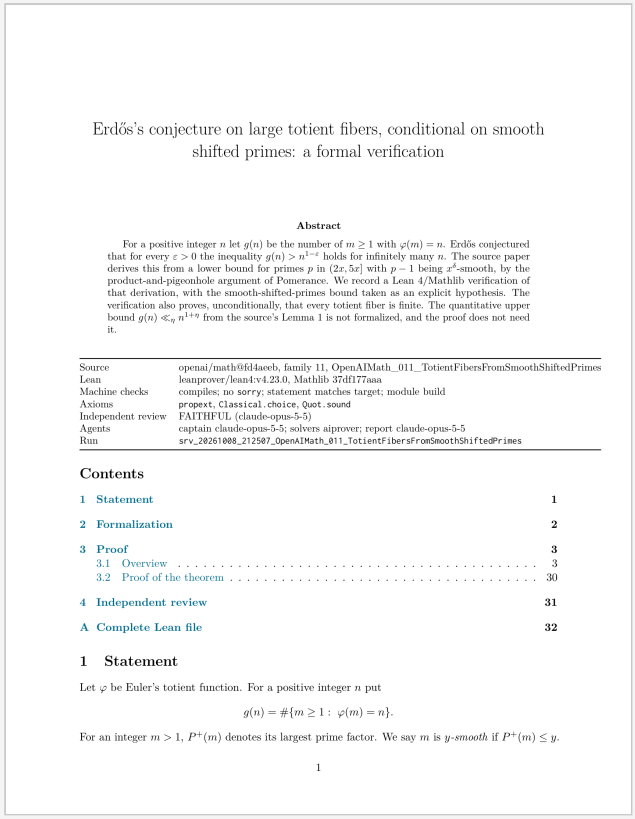
3 Proof of the theorem 9

A Correspondence with the formal proof 10

1 Setting

Throughout, φ is Euler's totient function on the nonnegative integers, with $\varphi(0) = 0$. All sets of integers below are subsets of $\mathbb{N} = \{0, 1, 2, \dots\}$. For $p \geq 1$ the symbol $p-1$ has its usual meaning. For a real number t we write

$$\lfloor t \rfloor_+ = \max(\lfloor t \rfloor, 0) \in \mathbb{N}$$



Erdős's conjecture on large totient fibers, conditional on smooth shifted primes: a formal verification

Abstract

For a positive integer n let $g(n)$ be the number of $m \geq 1$ with $\varphi(m) = n$. Erdős conjectured that for every $\varepsilon > 0$ the inequality $g(n) > n^{1-\varepsilon}$ holds for infinitely many n. The source paper derives this from a lower bound for primes p in $(2x, 5x]$ with $p-1$ being x^{δ}-smooth, by the product-and-pigeonhole argument of Pomerance. We record a Lean 4/Mathlib verification of that derivation, with the smooth-shifted-primes bound taken as an explicit hypothesis. The verification also proves, unconditionally, that every totient fiber is finite. The quantitative upper bound $g(n) \ll_\eta n^{1+\eta}$ from the source's Lemma 1 is not formalized, and the proof does not need it.

Source	openai/math@fd4aeeb, family 11, OpenAIMath_011_TotientFibersFromSmoothShiftedPrimes
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no `sorry`; statement matches target; module build
Axioms	`propext`, `Classical.choice`, `Quot.sound`
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Run	`srv_20261008_212507_OpenAIMath_011_TotientFibersFromSmoothShiftedPrimes`

Contents

1 Statement 1

2 Formalization 2

3 Proof 3
3.1 Overview . . . 3
3.2 Proof of the theorem . . . 30

4 Independent review 31

A Complete Lean file 32

1 Statement

Let φ be Euler's totient function. For a positive integer n put

$$g(n) = \#\{m \geq 1 : \varphi(m) = n\}.$$

For an integer $m > 1$, $P^+(m)$ denotes its largest prime factor. We say m is *y-smooth* if $P^+(m) \leq y$.

Figure 8. First pages of the informal proof (left) and the report (right) of OpenAI math problem 011, a conditional form of a conjecture of Erdős on large totient fibers. The remaining figures of this section and the next follow the same run.

Lemma 2.3 (Totient of a squarefree product)**.** *Let Q be a finite set of primes. Then*

$$\varphi\Big(\prod_{p \in Q} p\Big) = \prod_{p \in Q} (p-1).$$

Proof. We use induction on $|Q|$. The empty case reads $\varphi(1) = 1$.

Suppose $Q = Q' \cup \{p\}$ with $p \notin Q'$. Each $q \in Q'$ is a prime different from p, so q is coprime to p. Hence $\prod_{q \in Q'} q$ is coprime to p. By multiplicativity of φ, the induction hypothesis and $\varphi(p) = p - 1$,

$$\varphi\Big(p \prod_{q \in Q'} q\Big) = \varphi(p)\,\varphi\Big(\prod_{q \in Q'} q\Big) = (p-1) \prod_{q \in Q'} (q-1).$$

□

Lemma 3.3 (`totient_prod_primes`)**.** *Let $Q \subset \mathbb{N}$ be a finite set of primes. Then $\varphi(\prod_{p \in Q} p) = \prod_{p \in Q} (p-1)$.*

Lean: `totient_prod_primes` — proved by AIProver, job 1, session 1

```
theorem totient_prod_primes (Q : Finset ℕ) (hQ : ∀ p ∈ Q, p.Prime) :
    Nat.totient (∏ p ∈ Q, p) = ∏ p ∈ Q, (p - 1) := by
  induction' Q using Finset.induction_on with p Q hp ih
  · simp
  · have hp_prime : p.Prime := hQ p (Finset.mem_insert_self p Q)
    have hQ' : ∀ q ∈ Q, q.Prime := fun q hq => hQ q (Finset.mem_insert_of_mem hq)
    rw [Finset.prod_insert hp, Finset.prod_insert hp]
    have hcop : (∏ x ∈ Q, x).Coprime p := by
      refine Nat.Coprime.prod_left fun q hq => ?_
      have hq_prime : q.Prime := hQ' q hq
      refine hq_prime.coprime_iff_not_dvd.mpr fun h => ?_
      have hq_one_lt : 1 < q := hq_prime.one_lt
      have hp_eq_q : p = q := ((hp_prime.dvd_iff_eq hq_one_lt.ne').mp h)
      exact hp (hp_eq_q ▸ hq)
    rw [← mul_comm, Nat.totient_mul hcop, mul_comm, ih hQ', Nat.totient_prime hp_prime, mul_comm]
```

Proof. We use induction on Q. The empty case is $\varphi(1) = 1$.

For $Q \cup \{p\}$ with $p \notin Q$: the product $\prod_{q \in Q} q$ is coprime to p, because each $q \in Q$ is a prime different from p. By multiplicativity of φ, the induction hypothesis, and $\varphi(p) = p - 1$,

$$\varphi\Big(p \prod_{q \in Q} q\Big) = (p-1) \prod_{q \in Q} (q-1).$$

□

Figure 9. One lemma in both documents: the informal proof (left) and the report (right), where the Lean proof follows the statement.

The **Details** tab records the dates of the run and the models that it used. This information should accompany any citation of the result.

6 The Lean file

The Lean file contains the complete formal proof and can be checked independently by anyone with Lean. It is intended for a proof assistant rather than for a reader: the file of this run has 1,135 lines, and Lean has already verified each of its proofs. Consequently, a reader need only examine the final theorem and the definitions on which it depends.

```
theorem solution
    (hH : SmoothShiftedPrimesHypothesis) (ε : ℝ) (hε : 0 < ε) :
    Set.Infinite {n : ℕ | 0 < n ∧ ((totientFiberCard n : ℕ) : ℝ) > (n : ℝ) ^ (1
- ε)}
```

This statement asserts that for every $\varepsilon > 0$ there are infinitely many positive integers n with $g(n) > n^{1-\varepsilon}$, where $g(n)$ counts the solutions m of $\varphi(m) = n$. The assumption `hH` illustrates why the statement merits attention: the hypothesis on smooth shifted primes is assumed rather than proved, and the theorem is therefore conditional on it. A proved run contains no `sorry`, and the report lists the axioms on which the proof relies.

7 The replay

The replay presents the history of a run as a tree, read from top to bottom: the statement, its formalization, the plan, the lemmas, and the verified proof. As Figure 10 shows, it opens on the completed tree. The **Replay** tab of a run shows it within the record; **Open page** opens it on a page of its own, and **Full screen** enlarges it to the whole screen, and **Exit full screen** returns it to the record.

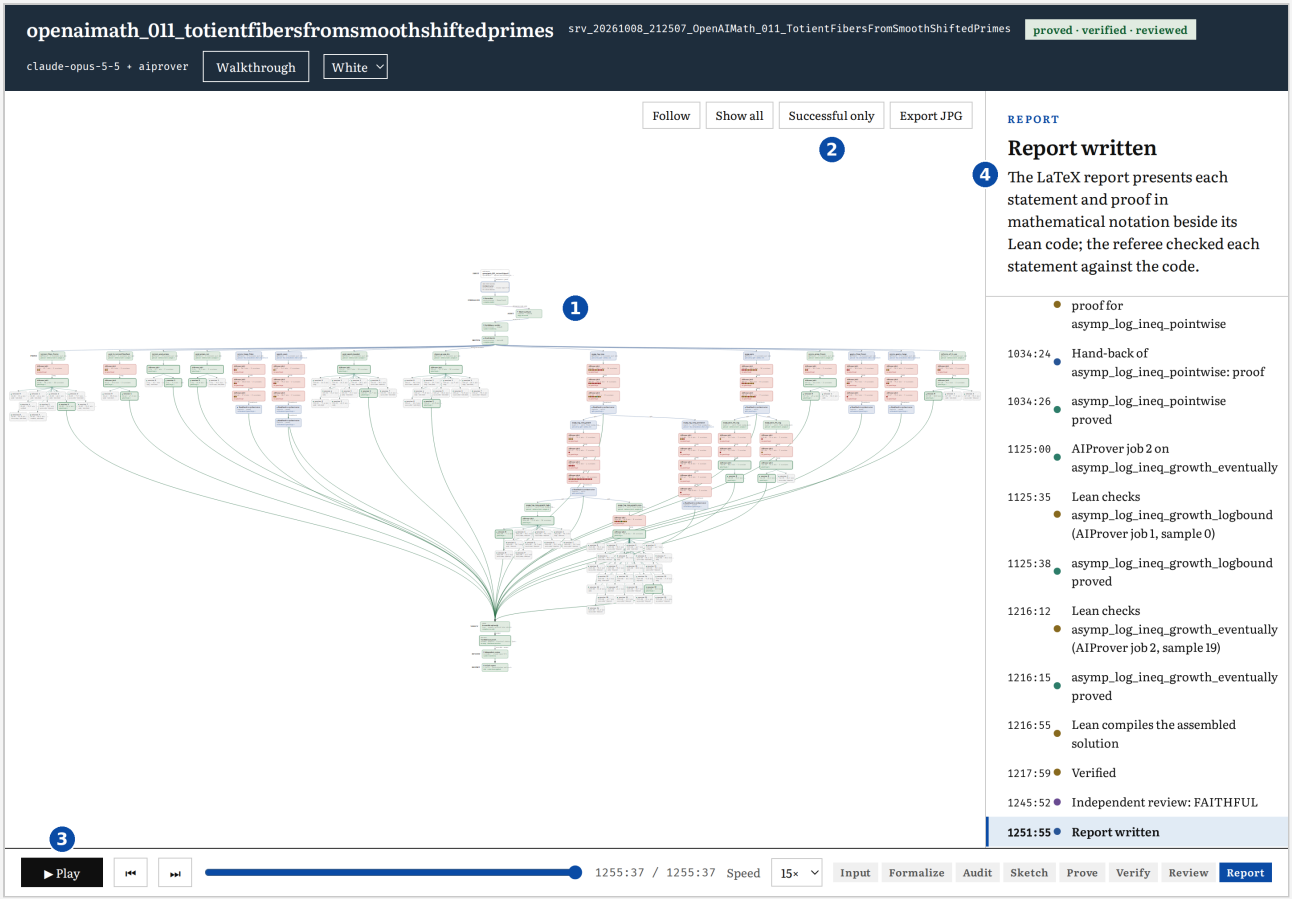


Figure 10. The replay.
1 Completed tree.
2 Successful only omits failed attempts.
3 Play reconstructs the tree step by step.
4 Description of the current step.

Selecting any node displays the full record of that step.

---

8 Blueprints

A blueprint presents a proof plan as a graph of statements, in which an arrow leads from each result to the results that use it. The **Blueprint** tab of a run shows the lemmas of its plan and marks each as planned, proved, or refuted. Selecting a lemma displays its statement in mathematical language, the idea of its proof, its Lean statement, and the lemmas that depend on it. **Outline** lists the same lemmas as text, and the blueprint can be downloaded in the LaTeX format of the leanblueprint package. **Full screen** enlarges the blueprint to the whole screen, which suits a large plan; **Exit full screen** or the Escape key returns it to the page.

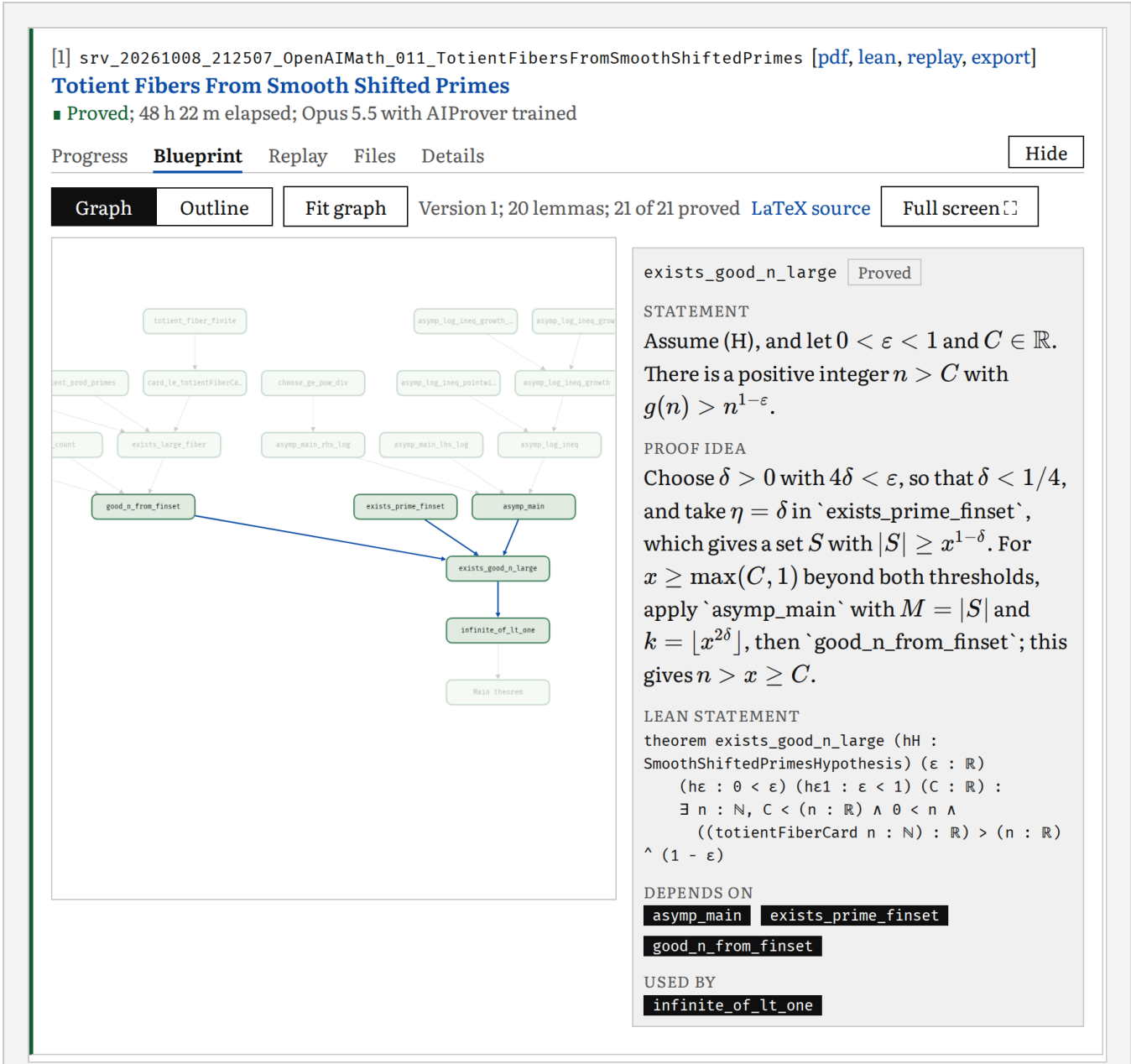


Figure 11. The Blueprint tab of the run of Figure 6, with `exists_good_n_large` selected. The graph highlights the lemma, the three lemmas it uses, and the lemma that uses it; the panel gives its statement, proof idea, and Lean statement.

A blueprint is updated as lemmas are proved. A plan is regarded as sound when each open lemma is plausible and its dependencies are appropriate, irrespective of how many lemmas have yet been proved.

The setting **Require review of the proof plan before proving** pauses a run before proving commences, so that a reader may approve the plan, approve it with comments, request a revision, or reject it. The run holds no computing resources during this pause.

A run imported from elsewhere may show Lean statements alone, since its natural-language text was not recorded.

9 Projects

A run addresses one result. A project addresses a body of connected results: the results are proved in dependency order on one shared Lean library, so that each may use those before it. A single theorem belongs in a run; a paper, or a theorem that requires several substantial preliminaries, belongs in a project.

Figure 12 shows the page of a project, with each of its controls numbered.

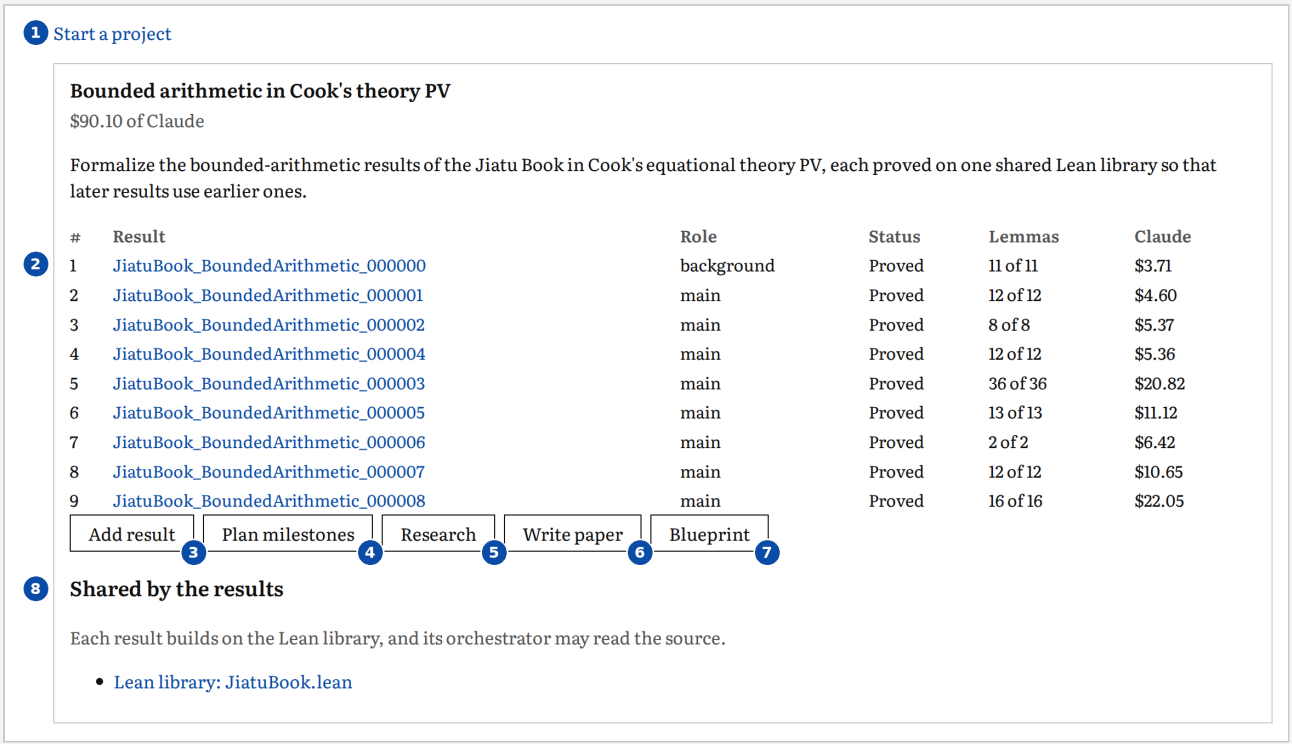


Figure 12. The page of a project, here the bounded-arithmetic results of the Jiatu Book.
1 Start a project opens a new project.
2 Results in dependency order, with status, lemmas, and cost; selecting one opens its run.
3 Add result writes a result by hand.
4 Plan milestones divides the goal into results.
5 Research puts a question to the project's model about a stalled result.
6 Write paper composes one article from the verified results.
7 Blueprint shows the results as a graph.
8 The Lean library and source files that the results share.

Starting a project

Select **Submit a problem**, then **Project** in place of **Single problem.** Enter a title, the goal, and the Lean version, GPU group, and prover that its runs will use. The orchestrator may be Claude or Codex, and the prover may be AIProver, Claude, Codex, or a server of your own. The project receives a new, empty library.

Signing in

A submission that uses Claude or Codex without a login opens that service's sign-in, which links to its own site. Claude shows a code to paste back into this page, and Codex shows a code here to enter on OpenAI's site. An API key is the alternative to either. The login stays in your own account on that machine, and this page does not keep a sign-in code.

Your own model server

Model servers, on the Submit page, adds any server with an OpenAI-compatible API. Enter its address, an API key if it needs one, and its kind: **AIProver** runs the AIProver harness on the server, and **General model** treats it as a chat model for any role. The server is checked before it is added, and it must answer from the machine that runs this page. Choose group `cpu` for a run that needs no shared model server. A key is stored on that machine and is never shown again.

Source files

A paper or notes may be attached as one file or as a zip of its files. LaTeX and other text files serve best, PDF files are harder to search, and office documents should first be converted to PDF or text. A short table of contents helps a large collection. An upload may hold up to 256 MB.

Source files are unverified reference material. **Plan milestones**, **Research**, and **Ask Claude** read them, and the orchestrator of each run may open them. The solvers do not receive them, and **Write paper** uses only verified results. The orchestrator decides whether to read a file, so text that a run must use belongs in the goal or in the statement of a result. Uploaded files neither create results nor add to the library, and a Lean file among them cannot be imported by a run. Each file may be read from the project's **Shared by the results** section.

As an example, consider a project with the goal of proving that every integer greater than 1 factors uniquely into primes. The sections below follow this project.

Planning the results

Plan milestones asks the project's model to divide the goal into results, using the source if one is attached, and a second session referees the proposal. The example might receive four results: Bezout's identity, Euclid's lemma, the existence of a prime factorization, and uniqueness. **Background** marks an established result on which the others build, and **main** marks a contribution of the project.

Nothing runs until **Start remaining results** is selected. Until then, each result may be selected and its name, statement, informal proof, and dependencies edited; it may also be moved or removed. **Add result** writes one by hand. The dependency order deserves particular care, since it reflects the model's reading of the source.

Following the project

The table lists each result with its status, lemmas, and cost. Selecting a result opens the record of its run in place, with the tabs of Section 4. The **Blueprint** button shows the project as a graph of its results, as Figure 13 shows. Each result that has a run offers its own blueprint of lemmas, so that the project may be read at either level.

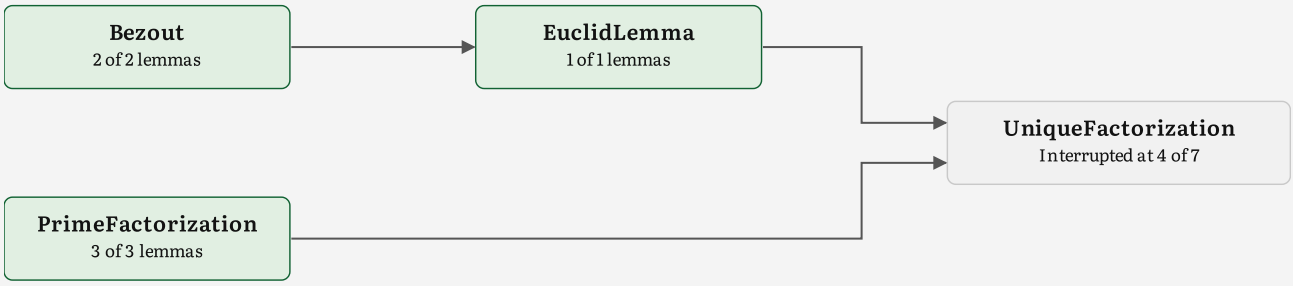


Figure 13. Blueprint of the example project. Green results are proved. A result may use those from which an arrow leads to it.

Shared by the results lists what the runs have in common: the Lean library, which grows as each result is verified, and the source files.

Research on open lemmas

When a result stalls, **Research** puts a question to the project's model within a spending limit, and a second session referees the answer. The report may add **open lemmas**, which remain unproved, and **claims**, on which the project relies without a Lean proof. To use a report, select **Resume** on the stalled run and choose the report under **Project research**; the run then replans with the findings and retains its proved lemmas.

> **Reports are not proofs.** A research report guides the plan, and every result must still pass Lean. An open lemma is never assumed.

Reviewing claims

Review of a claim is optional, and a project does not require an expert. **Confirm** and **Reject** record a person's decision. **Ask Claude** records an independent opinion, which is not a proof and is shown apart from any person's decision. **Prove in Lean** adds the claim to the project as a result, and is the only choice that settles it.

Figure 14 shows these sections for the example project.

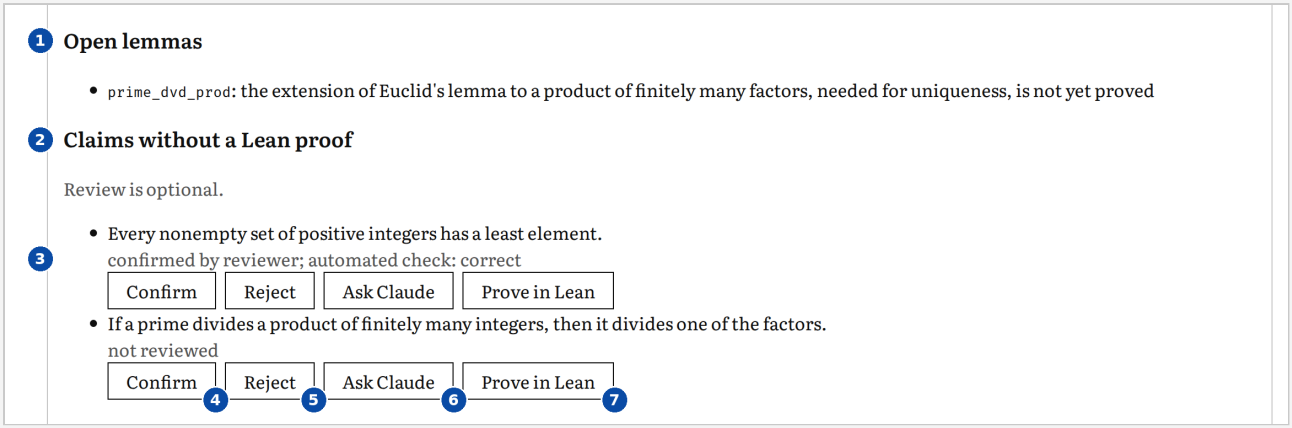


Figure 14. Open lemmas and claims of the example project.
1 Open lemmas: statements that remain unproved, each with its reason; a run never assumes one.
2 Claims without a Lean proof: statements on which the project relies without a Lean proof.
3 Review status: a person's decision and, once requested, an automated check.
4 Confirm records that a person accepts the claim.
5 Reject records that a person rejects it.
6 Ask Claude requests an independent check, which is not a proof.
7 Prove in Lean adds the claim to the project as a result, the only choice that settles it.

Writing the paper

Once each result has its documents, which **Generate documents** on a run produces, **Write paper** composes one article from them and checks it against the Lean statements. Before any result has a run, **Write paper** instead writes a proposal from the latest milestone plan for the expert's feedback: the question, the approach, the main statements with proof ideas, the roadmap, and the decisions left to the author. Nothing in it is machine-verified.

---

10 Incomplete runs

An unfinished run retains its proved lemmas. **Resume**, at the right of the record's tabs, continues the run from the point at which it stopped. The replay identifies the lemmas that remain unproved, and the appropriate response depends on their content.

- **A lemma appears to be false.** The statement may require an additional hypothesis; a corrected statement should be submitted as a new run.
- **The plan appears unsuitable.** A new run with a more detailed informal proof may succeed.
- **The plan appears sound.** The run should be resumed.

The **Unfinished** tab of the main page collects these runs.

11 Glossary

Lean	A language and proof assistant in which mathematics is written so that a computer can verify it.
Mathlib	The shared library of formalized mathematics on which these proofs build.
Formal statement	A theorem as written in Lean.
Formalization	The translation of mathematics into Lean.
Orchestrator	The agent that formalizes the problem, plans the proof, and resolves difficulties.
Auditor	The agent that checks a formalization without access to the source text.
Solver	A model, AIProver, that proves individual lemmas.
Sorry	Lean's placeholder for a missing proof. A proved run contains none.
Hand-back	The return of a lemma from the solvers to the orchestrator after repeated failure.
Blueprint	A proof plan shown as a graph of statements and their dependencies.
Project	Connected results proved in dependency order on one shared library.
Milestone	One result of a project, proved by its own run.
Claim	A statement on which a project relies without a Lean proof.
Open lemma	A statement that a project needs and that remains unproved; it is recorded with its reason and never assumed.